CENG 476 - INTRODUCTION TO DEEP LEARNING

Plant Disease Classification Using Deep Learning and Transfer Learning

FINAL PROJECT REPORT

Student Name: Emir EVREN

Student ID: 210444038

Course: CENG 476 - Introduction to Deep Learning

Framework: PyTorch 2.11.0 + CUDA 13.0

Date: August 2026

Abstract

Automated plant disease recognition can support faster and more accessible crop-health assessment, but model quality depends on both discriminative performance and computational practicality. This study develops and evaluates a complete PyTorch pipeline for 38-class plant condition classification using 54,305 RGB leaf images from the PlantVillage dataset. A compact four-block convolutional neural network (CNN) was established as a train-from-scratch baseline. Two ImageNet-pretrained architectures, ResNet18 and EfficientNet-B0, were then fine-tuned end-to-end with differential learning rates. The training strategy combined data augmentation, batch normalization, dropout, AdamW weight decay, ReduceLROnPlateau scheduling, early-stopping logic, mixed-precision training, validation Macro-F1 checkpoint selection, and a controlled three-rate dropout sensitivity experiment. On the locked 5,431-image test set, the baseline achieved 87.42% accuracy and 0.8201 Macro-F1. ResNet18 improved these results to 99.26% and 0.9869, while EfficientNet-B0 reached 99.52% and 0.9923 using approximately 63.8% fewer parameters than ResNet18. A validation-tuned 50:50 soft-voting ensemble further improved test accuracy to 99.76% and Macro-F1 to 0.9950. Grad-CAM analysis indicated that the strongest EfficientNet-B0 decisions generally emphasized leaf lesions, texture, and venation, while errors concentrated in visually similar disease pairs and rare healthy classes. The results demonstrate the value of transfer learning and model complementarity on controlled leaf imagery, while also highlighting that PlantVillage performance should not be interpreted as guaranteed field robustness.

Keywords: plant disease classification; PlantVillage; transfer learning; ResNet18; EfficientNet-B0; soft voting; Grad-CAM; PyTorch

Contents

Introduction

Plant diseases reduce crop quality and yield, yet timely diagnosis may be difficult when specialist support is limited. Image-based deep learning offers a scalable route for recognizing visible symptoms from leaf photographs. The PlantVillage initiative released more than 50,000 curated images to support this direction [1], and subsequent work demonstrated that convolutional neural networks can achieve very high accuracy when train and test images originate from the same controlled domain [3].

This project addresses a single-label, 38-class classification task in which each RGB image is mapped to one crop-condition class. The study follows a controlled development sequence: a custom baseline CNN, learning-rate pilots, a dropout ablation, full fine-tuning of ResNet18 and EfficientNet-B0, locked test evaluation, validation-tuned model ensembling, and Grad-CAM interpretability analysis. This sequence enables a direct comparison of model capacity, transfer learning, regularization, optimization behavior, and interpretability under a common evaluation protocol.

Objectives and contributions

- Build a reproducible PyTorch data pipeline with deterministic, stratified validation and test subsets.
- Establish a compact CNN baseline containing Conv-BN-ReLU-Pool blocks and compare three baseline learning rates.
- Compare baseline dropout rates 0.20, 0.40, and 0.60 under fixed three-epoch pilot conditions without using the test set.
- Fine-tune ImageNet-pretrained ResNet18 and EfficientNet-B0 models using architecture-specific classifiers and differential learning rates.
- Evaluate all selected checkpoints on a locked test set with accuracy, precision, recall, Macro-F1, Weighted-F1, Top-3 accuracy, ROC-AUC, and confusion matrices.
- Evaluate a validation-tuned ResNet18 + EfficientNet-B0 soft-voting ensemble as an additional model-combination experiment.
- Use Grad-CAM to investigate whether the final single model relies on disease-relevant leaf regions.

> Main outcome: EfficientNet-B0 was the best single model, and the equal-weight ResNet18 + EfficientNet-B0 ensemble was the best overall prediction system.

Related Work and Technical Background

Deep learning for PlantVillage

Hughes and Salathé introduced the PlantVillage image repository to encourage machine-learning research for mobile plant-health diagnostics [1]. Mohanty, Hughes, and Salathé later trained deep CNNs on 54,306 PlantVillage images and reported 99.35% held-out accuracy under the controlled dataset protocol [3]. Their cross-domain experiment, however, fell to 31.4% accuracy on images gathered from different online sources. This contrast is important: strong same-domain scores establish classification feasibility, but they do not prove deployment robustness.

Transfer-learning architectures

ResNet introduced residual skip connections that allow networks to learn residual mappings and mitigate optimization degradation in deeper models [4]. ResNet18 provides a practical accuracy-cost balance for limited hardware. EfficientNet uses compound scaling to coordinate network depth, width, and input

resolution, and its MBConv blocks offer strong parameter efficiency [5]. Both architectures are appropriate for transfer learning because their ImageNet-pretrained filters provide general visual features that can be adapted to leaf texture, color, shape, and lesion patterns.

Regularization and interpretability

Batch normalization stabilizes intermediate activation distributions and supports efficient optimization [6]. Dropout randomly removes activations during training to reduce co-adaptation [7], while AdamW decouples weight decay from the adaptive gradient update [8]. For interpretability, Grad-CAM uses class-specific gradients flowing into a convolutional layer to generate coarse localization maps [9]. These techniques were integrated into the project rather than treated as post-hoc checklist items.

Dataset and Problem Formulation

Dataset characteristics

The Kaggle PlantVillage mirror specified in the project proposal was used as the data source [2]. The local copy contains 54,305 RGB images distributed over 38 crop-condition classes from 14 crop species. Twelve classes represent healthy leaves and the remaining classes represent disease or pest conditions. The task is multi-class and single-label: for an input image x, the model produces 38 logits, and the predicted class is the index with the maximum posterior probability.

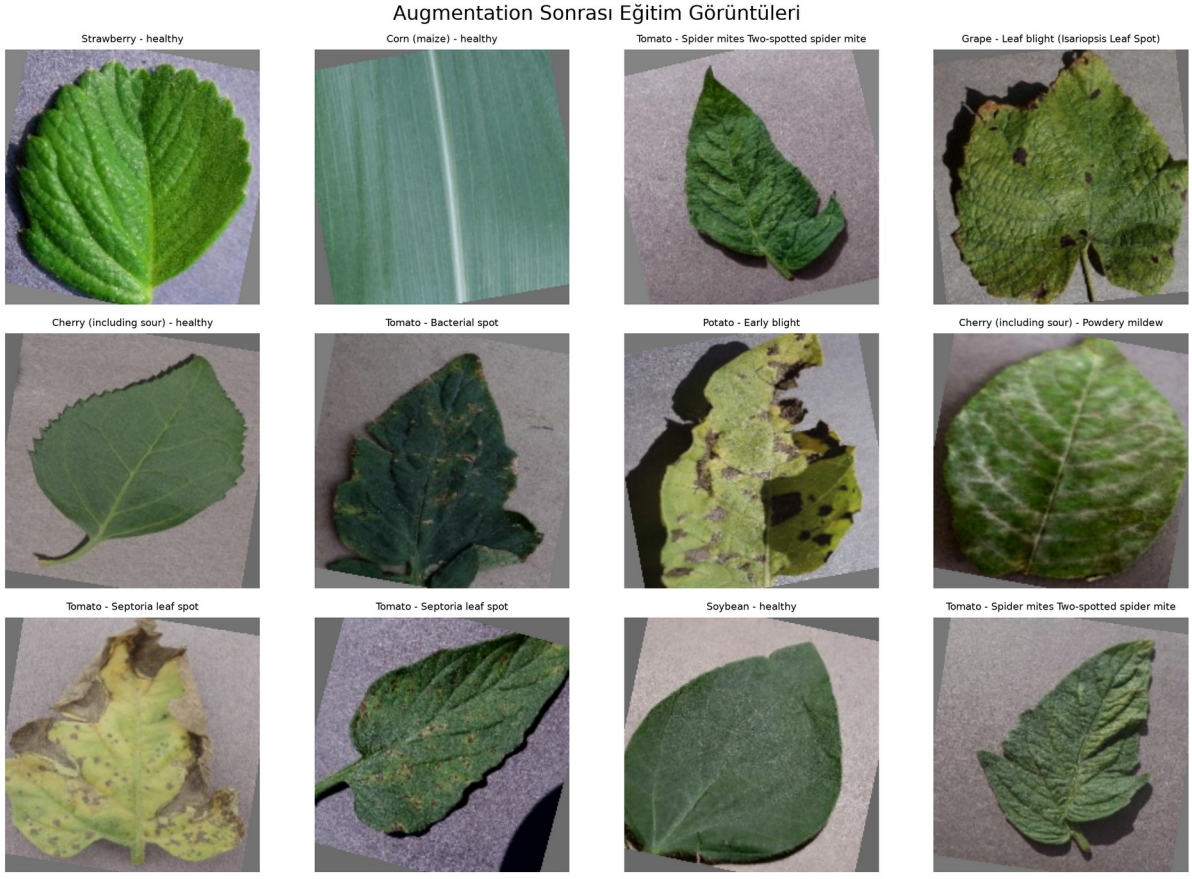

Figure 1. Representative training images after stochastic augmentation. The operations preserve disease-relevant structure while varying crop, orientation, color, and scale.

Split	Images	Share	Purpose
Training	43,444	80.00%	Optimization and augmentation
Validation	5,430	10.00%	Checkpoint and ensemble-weight selection
Test	5,431	10.00%	Locked final evaluation only
Total	54,305	100.00%	38 classes

Table 1. Dataset split and evaluation purpose.

Stratification and class imbalance

The Kaggle training directory was retained as the 43,444-image training set. The original 10,861-image validation directory was divided equally into validation and test subsets using stratified sampling with random seed 42. Programmatic checks confirmed matching class-to-index mappings and no overlap between the resulting validation and test indices. The largest class contains 5,507 images, whereas Potato healthy contains only 152 images, yielding an imbalance ratio above 36:1. Therefore, Macro-F1 was used as the primary model-selection metric so that rare classes contributed equally to checkpoint decisions.

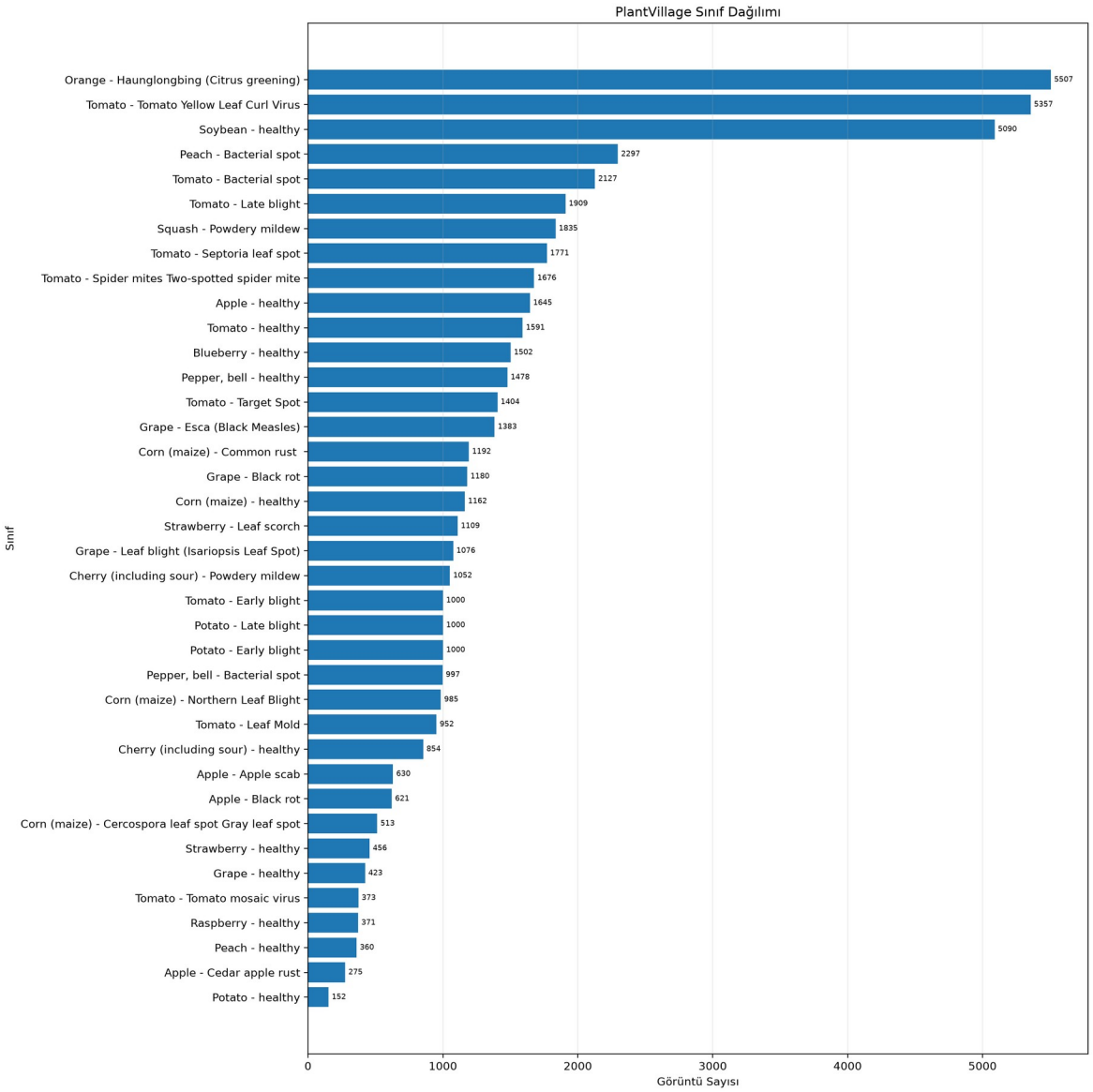

Figure 2. PlantVillage class distribution. The dataset is strongly imbalanced, motivating Macro-F1 monitoring in addition to accuracy.

Proposed Method

Baseline CNN

The baseline was intentionally compact and trained from scratch. Four feature blocks each apply a 3 × 3 convolution, batch normalization, ReLU activation, and 2 × 2 max pooling. Channel depth increases from 32 to 256 while spatial resolution decreases from 224 × 224 to 14 × 14. Adaptive average pooling converts the final feature tensor to 256 × 1 × 1, followed by dropout (p = 0.40) and a 256-to-38 linear classifier. This design contains 399,142 trainable parameters and provides a controlled benchmark for the transfer-learning models. The corresponding architecture diagram is included with the project figures.

Stage	Output shape	Operation
Input	3 × 224 × 224	RGB image

Stage	Output shape	Operation
Block 1	32 × 112 × 112	Conv3×3 - BN - ReLU - MaxPool
Block 2	64 × 56 × 56	Conv3×3 - BN - ReLU - MaxPool
Block 3	128 × 28 × 28	Conv3×3 - BN - ReLU - MaxPool
Block 4	256 × 14 × 14	Conv3×3 - BN - ReLU - MaxPool
Pooling	256 × 1 × 1	Adaptive average pooling
Classifier	38 logits	Flatten - Dropout(0.40) - Linear

Table 2. Baseline CNN architecture.

ResNet18 transfer learning

ResNet18 was initialized with the default Torchvision ImageNet weights. Its stem convolution and max-pooling stage feed four residual stages containing [2, 2, 2, 2] BasicBlocks with channel dimensions 64, 128, 256, and 512. Native batch-normalization layers and ReLU activations were retained. The original 1,000-class fully connected layer was replaced by Dropout(p = 0.30) and Linear(512, 38). All layers remained trainable. The resulting model contains 11,196,006 parameters.

EfficientNet-B0 transfer learning

EfficientNet-B0 was also initialized with default ImageNet weights and fine-tuned end-to-end. Its feature extractor combines a convolutional stem with mobile inverted bottleneck (MBConv) blocks, depthwise convolutions, squeeze-and-excitation, batch normalization, and SiLU activations. Global average pooling yields a 1,280-dimensional feature vector. The classifier was replaced by Dropout(p = 0.30) and Linear(1280, 38). The model contains 4,056,226 parameters, substantially fewer than ResNet18.

Model	Feature extractor	New classifier	Parameters
ResNet18	Residual BasicBlocks; ReLU	Dropout 0.30 + Linear 512 → 38	11,196,006
EfficientNet-B0	MBConv + squeeze-excitation; SiLU	Dropout 0.30 + Linear 1280 → 38	4,056,226

Table 3. Transfer-learning architecture summary.

Activation and output design

The baseline and ResNet18 use ReLU in their hidden feature blocks, whereas EfficientNet-B0 uses SiLU inside its MBConv backbone. The final layer of every model returns unnormalized logits. CrossEntropyLoss internally combines log-softmax with negative log likelihood, avoiding a redundant softmax during training. Softmax probabilities are computed only for ranking, ROC-AUC calculation, Top-3 accuracy, soft voting, and Grad-CAM confidence reporting.

Soft-voting ensemble

The ensemble combines the posterior probabilities of the selected ResNet18 and EfficientNet-B0 checkpoints. Candidate EfficientNet weights of 0.25, 0.50, and 0.75 were evaluated only on the validation set; pure single-model endpoints were retained as references but were ineligible for selection. Candidate ranking used validation Macro-F1, with validation negative log likelihood as a tie-breaker. The best candidate assigned equal weight to both models.

$$p_{ensemble}(x) = 0.50 \cdot p_{ResNet18}(x) + 0.50 \cdot p_{EfficientNet\text{-}B0}(x)$$

Grad-CAM explainability

Grad-CAM was applied to the last EfficientNet-B0 feature stage (model.features[-1]). Six correctly classified examples from distinct classes and six error cases from distinct true/predicted class pairs were selected. For errors, separate maps were generated for the predicted and true classes. The analysis is post-hoc: it does not alter training or accuracy, and the heatmaps should be interpreted as coarse class-discriminative evidence rather than pixel-level segmentation.

Experimental Setup

Preprocessing and augmentation

All inputs were converted to 224 × 224 tensors and normalized with the ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. Training augmentation consisted of a random resized crop with scale 0.80-1.00, horizontal flipping with probability 0.50, random rotation within ±15 degrees using bilinear interpolation and gray fill, and color jitter with brightness, contrast, and saturation amplitudes of 0.20. Validation and test images used a deterministic resize to 256 pixels followed by a 224 × 224 center crop.

Optimization, scheduling, and regularization

Model	Batch	Backbone LR	Head LR	Weight decay	Dropout	Max epochs	ES patience
Baseline CNN	64	5×10^{-4}	5×10^{-4}	1×10^{-4}	0.40	15	6
ResNet18	32	1×10^{-4}	5×10^{-4}	1×10^{-4}	0.30	12	5
EfficientNet-B0	32	1×10^{-4}	5×10^{-4}	1×10^{-4}	0.30	12	5

Table 4. Final training hyperparameters.

AdamW used $\beta_1 = 0.9$ and $\beta_2 = 0.999$ with unweighted cross-entropy. Transfer-learning models used differential learning rates: the pretrained backbone was updated at 1×10^{-4} while the new classifier was updated at 5×10^{-4}. ReduceLROnPlateau monitored validation loss, halved the learning rate after two non-improving epochs, and enforced a minimum rate of 1×10^{-6}. Checkpoints were selected by validation Macro-F1; a loss improvement resolved near-ties within 1×10^{-4}. Early stopping patience was six epochs for the baseline and five for transfer models. In the final fixed-length runs, the planned epoch limit was reached before the early-stopping condition, but the best-checkpoint logic prevented later degradation from replacing a stronger model.

Hardware, software, and reproducibility

Component	Configuration
Operating system	Windows
Python	3.14.3
PyTorch / Torchvision	2.11.0+cu130 / 0.26.0+cu130
CUDA runtime	13.0
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GB
Mixed precision	Enabled with torch.autocast and GradScaler
Random seed	42 for Python, NumPy, PyTorch, and CUDA
DataLoader workers	Training: 2; validation/test: 0

Table 5. Execution environment.

CUDA deterministic mode was enabled and cuDNN benchmarking was disabled. Automatic mixed precision reduced memory consumption. A Windows multiprocessing/pagefile failure initially occurred when evaluation workers imported PyTorch, SciPy, and scikit-learn in child processes. The final stable configuration used two workers for training and zero workers for clean-train, validation, and test evaluation. Before full experiments, a 38-image sanity test (one image per class, no augmentation, dropout, or weight decay) reached 100% accuracy by step 40, supporting the correctness of the model, label, and optimization pipeline.

Evaluation metrics

The report includes accuracy, macro precision, macro recall, Macro-F1, Weighted-F1, Top-3 accuracy, and one-vs-rest macro and weighted ROC-AUC. Macro metrics average class-level scores uniformly and are therefore central for the imbalanced dataset. Weighted-F1 additionally reflects class support. Confusion matrices and per-class F1 scores expose failure modes hidden by aggregate accuracy. ROC-AUC values are reported with higher precision because rounding to four decimals can display near-perfect values as 1.0000. The final EfficientNet-B0 checkpoint was also used to generate a macro/micro one-vs-rest ROC curve; the measured macro-average ROC-AUC was 0.999986 and the micro-average ROC-AUC was 0.999990. A macro/micro one-vs-rest ROC curve was generated for the selected checkpoint and retained with the project figures.

Results and Discussion

Baseline learning-rate pilots

Learning rate	Best epoch	Val. accuracy	Val. Macro-F1	Observation
1×10^{-3}	2	62.28%	0.5250	Peaked early, then declined
5×10^{-4}	3	63.90%	0.4953	Stable upward trend
3×10^{-4}	3	61.25%	0.4803	Stable but slower

Table 6. Three-epoch baseline learning-rate pilots.

Although 1×10^{-3} produced the highest short-pilot Macro-F1 at epoch 2, its validation score declined at epoch 3 and its validation loss was more unstable. The 5×10^{-4} pilot maintained the strongest epoch-3 accuracy and a monotonic Macro-F1 increase, so it was selected as a more conservative full-run rate. A separate controlled dropout sensitivity experiment then compared p = 0.20, 0.40, and 0.60 while holding learning rate (5×10^{-4}), batch size (64), weight decay (1×10^{-4}), and the three-epoch budget fixed. Best validation Macro-F1 values were 0.4990, 0.4953, and 0.4311, respectively. The 0.20 and 0.40 settings were therefore very close over this short pilot (difference 0.0037), whereas 0.60 clearly reduced short-run performance and produced the highest validation loss (1.9552). Because this ablation was conducted as a post-hoc sensitivity study after the original final baseline protocol had already been fixed, it was not used to retroactively replace the reported 0.40 full-run checkpoint. A longer multi-seed sweep would be required before making that change. The saved pilot histories and comparison plot document the same trend.

Training dynamics

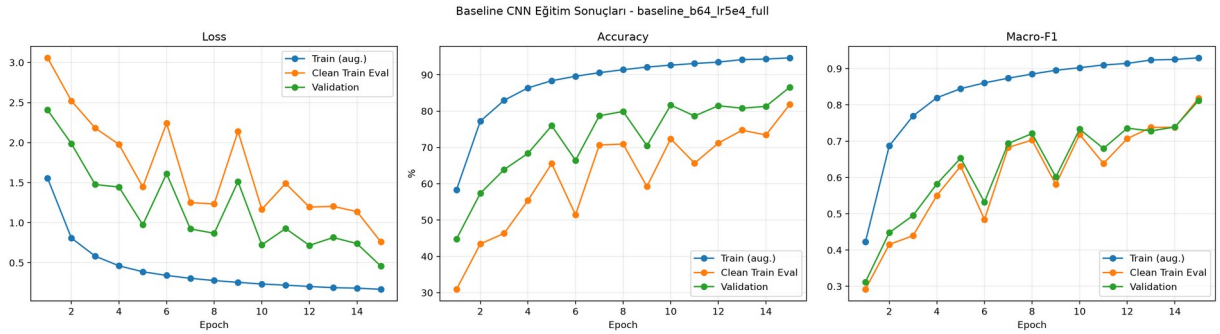

Figure 3. Baseline training, clean-train evaluation, and validation curves. Validation performance improves overall but remains noisy and well below the augmented-training trajectory.

The baseline gradually improved to a best validation Macro-F1 of 0.8121 at epoch 15. Its augmented-training Macro-F1 reached 0.9296, indicating a material generalization gap. The balanced 760-image clean-train subset also fluctuated because it contains only 20 examples per class. Data augmentation and weight decay limited direct memorization, but the small baseline still lacked the pretrained visual representation needed for uniformly strong rare-class performance.

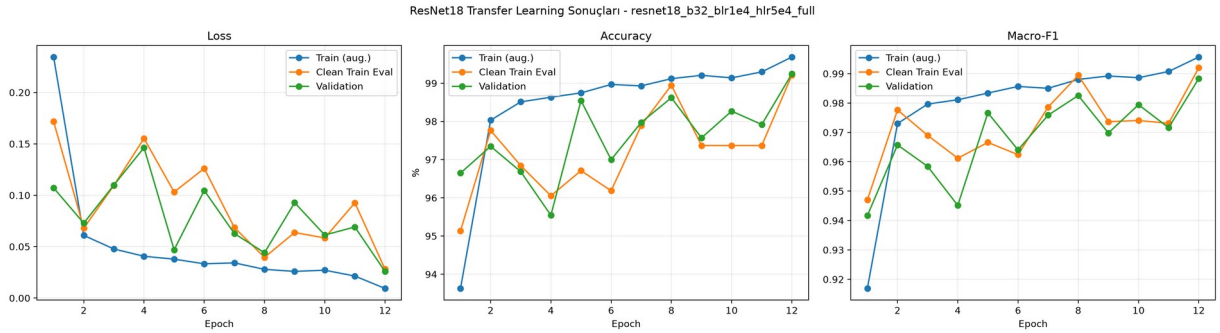

Figure 4. ResNet18 transfer-learning curves. The best checkpoint occurs at epoch 12 after the backbone learning rate is reduced to 5×10^{-5}.

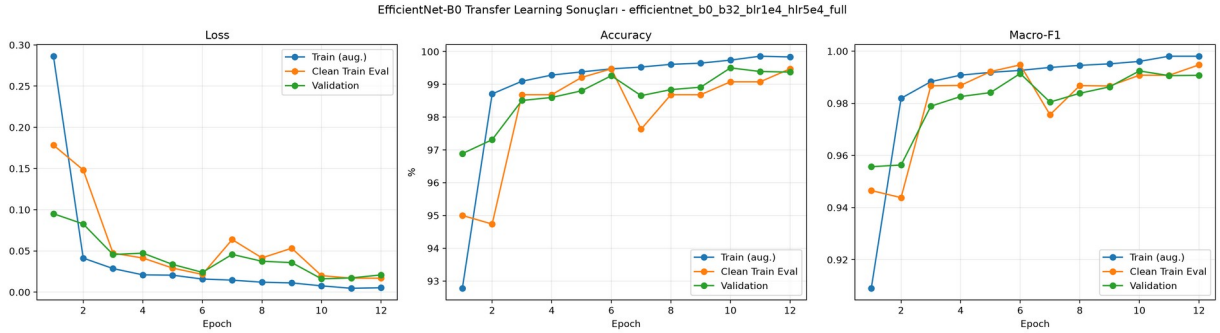

Figure 5. EfficientNet-B0 transfer-learning curves. Validation Macro-F1 peaks at epoch 10, and the stored checkpoint preserves that state despite small later fluctuations.

Both transfer-learning models converged rapidly, exceeding 0.94 validation Macro-F1 after the first epoch. ResNet18 reached its best validation Macro-F1 of 0.9883 at epoch 12. EfficientNet-B0 peaked at 0.9924 at epoch 10; its epoch-11 and epoch-12 validation scores were slightly lower even though augmented-training performance continued to increase. This divergence is mild overfitting, and it confirms the importance of validation-based checkpoint selection.

Locked test-set performance

Model	Loss	Accuracy	Macro-P	Macro-R	Macro-F1	Weighted-F1
Baseline CNN	0.4627	87.42%	0.8729	0.8060	0.8201	0.8711
ResNet18	0.0248	99.26%	0.9899	0.9855	0.9869	0.9927
EfficientNet-B0	0.0163	99.52%	0.9932	0.9917	0.9923	0.9952
Ensemble	0.0150	99.76%	0.9963	0.9941	0.9950	0.9976

Table 7. Locked test-set results. Bold interpretation in the text identifies the best single model and best overall system.

Model	Errors	Top-3 accuracy	Macro ROC-AUC	Weighted ROC-AUC
Baseline CNN	683	95.75%	0.99630	0.99707
ResNet18	40	99.96%	0.99998	0.99999
EfficientNet-B0	26	99.98%	0.99998	0.99999
Ensemble	13	99.98%	0.99998	0.99999

Table 8. Additional locked test-set metrics.

The baseline made approximately 683 errors, whereas ResNet18 and EfficientNet-B0 reduced this to 40 and 26 errors. EfficientNet-B0 was the best single model with 99.52% accuracy and 0.9923 Macro-F1. The ensemble further reduced the error count to 13 and achieved 99.76% accuracy and 0.9950 Macro-F1. Relative to EfficientNet-B0 alone, this corresponds to a 50% reduction in test errors, although inference now requires both networks.

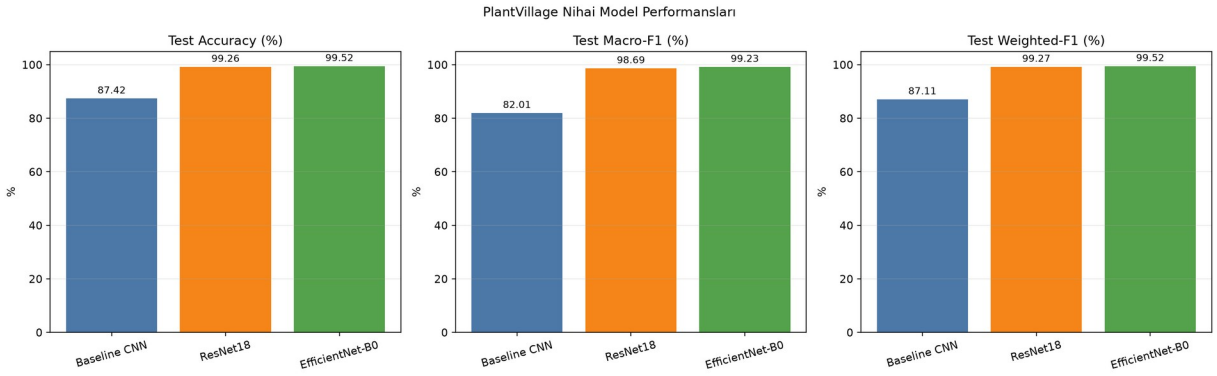

Figure 6. Test accuracy, Macro-F1, and Weighted-F1 for the three individual models. Transfer learning produces a large gain over the train-from-scratch baseline.

Accuracy-efficiency trade-off

Model	Parameters	Training time	Peak GPU memory	Accuracy	Macro-F1
Baseline CNN	0.399 M	145.4 min	1301.1 MB	87.42%	0.8201
ResNet18	11.196 M	69.8 min	574.2 MB	99.26%	0.9869
EfficientNet-B0	4.056 M	65.4 min	1461.9 MB	99.52%	0.9923
Ensemble	15.252 M combined	No retraining	Not separately logged	99.76%	0.9950

Table 9. Computational cost and predictive performance.

EfficientNet-B0 uses approximately 63.8% fewer parameters than ResNet18 while achieving higher test accuracy and Macro-F1. Its peak allocated GPU memory was higher in this implementation because MBConv activation patterns and the specific training graph differ from ResNet18; parameter count alone does not determine activation memory. ResNet18 trained in 69.8 minutes, EfficientNet-B0 in 65.4 minutes, and the baseline in 145.4 minutes on the recorded hardware. The baseline's longer run reflects 15 epochs and a larger batch, as well as the absence of pretrained features.

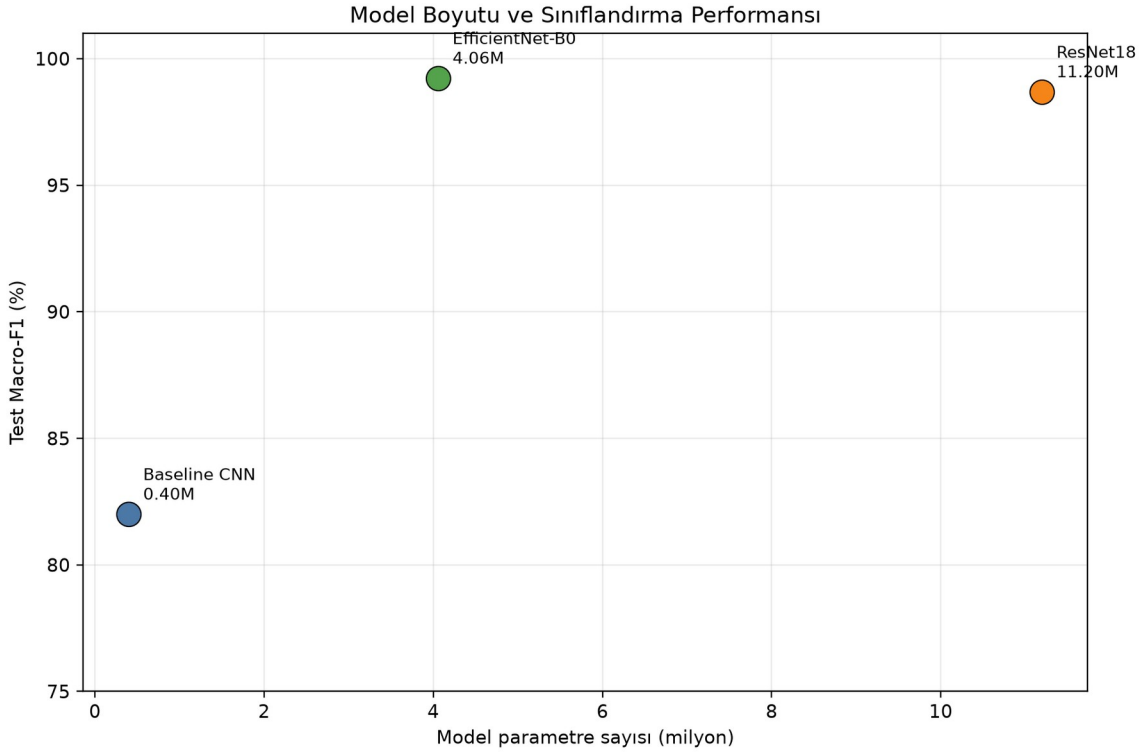

Figure 7. Parameter count versus test Macro-F1 for the three individual models. EfficientNet-B0 provides the strongest single-model trade-off.

Ensemble analysis

ResNet w.	EffNet w.	Status	Val. loss	Val. accuracy	Val. Macro-F1
1.00	0.00	Reference	0.0259	99.24%	0.9883
0.75	0.25	Eligible	0.0151	99.41%	0.9907
0.50	0.50	Eligible	0.0128	99.61%	0.9936
0.25	0.75	Eligible	0.0124	99.52%	0.9923
0.00	1.00	Reference	0.0161	99.50%	0.9924

Table 10. Validation-only ensemble weight search. The 0.50/0.50 candidate was selected by Macro-F1.

Equal weighting achieved validation Macro-F1 0.9936, exceeding both pure checkpoints and the other eligible mixtures. The test set was not used during this selection. The ensemble's improvement shows that the two high-accuracy networks still make partially complementary errors. Its disadvantage is inference cost: both networks must be stored and executed, so EfficientNet-B0 remains preferable when latency, memory, or deployment simplicity is the primary constraint.

Class-level and error analysis

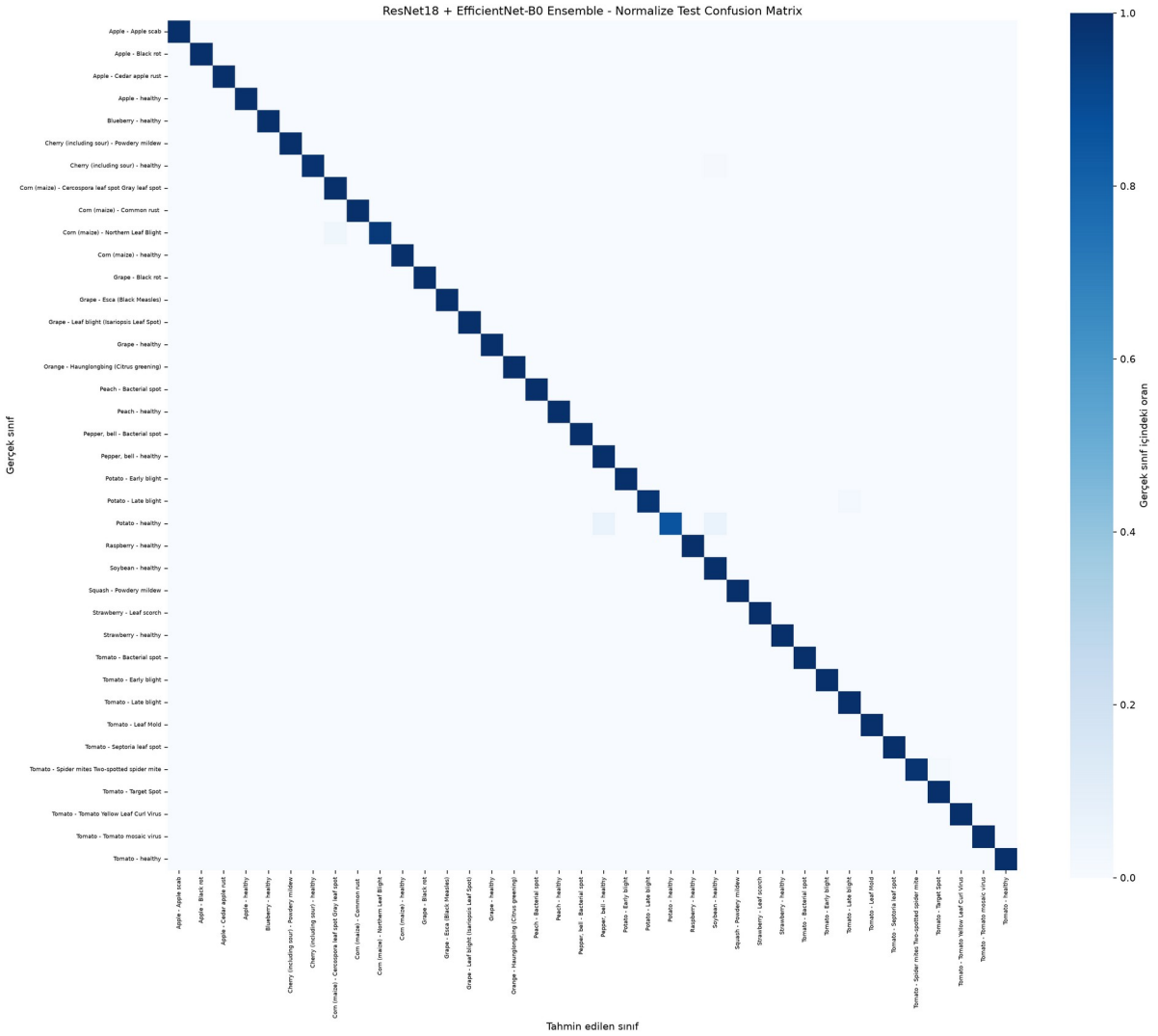

Figure 8. Row-normalized ensemble confusion matrix. The dark diagonal reflects correct classification for nearly all test examples.

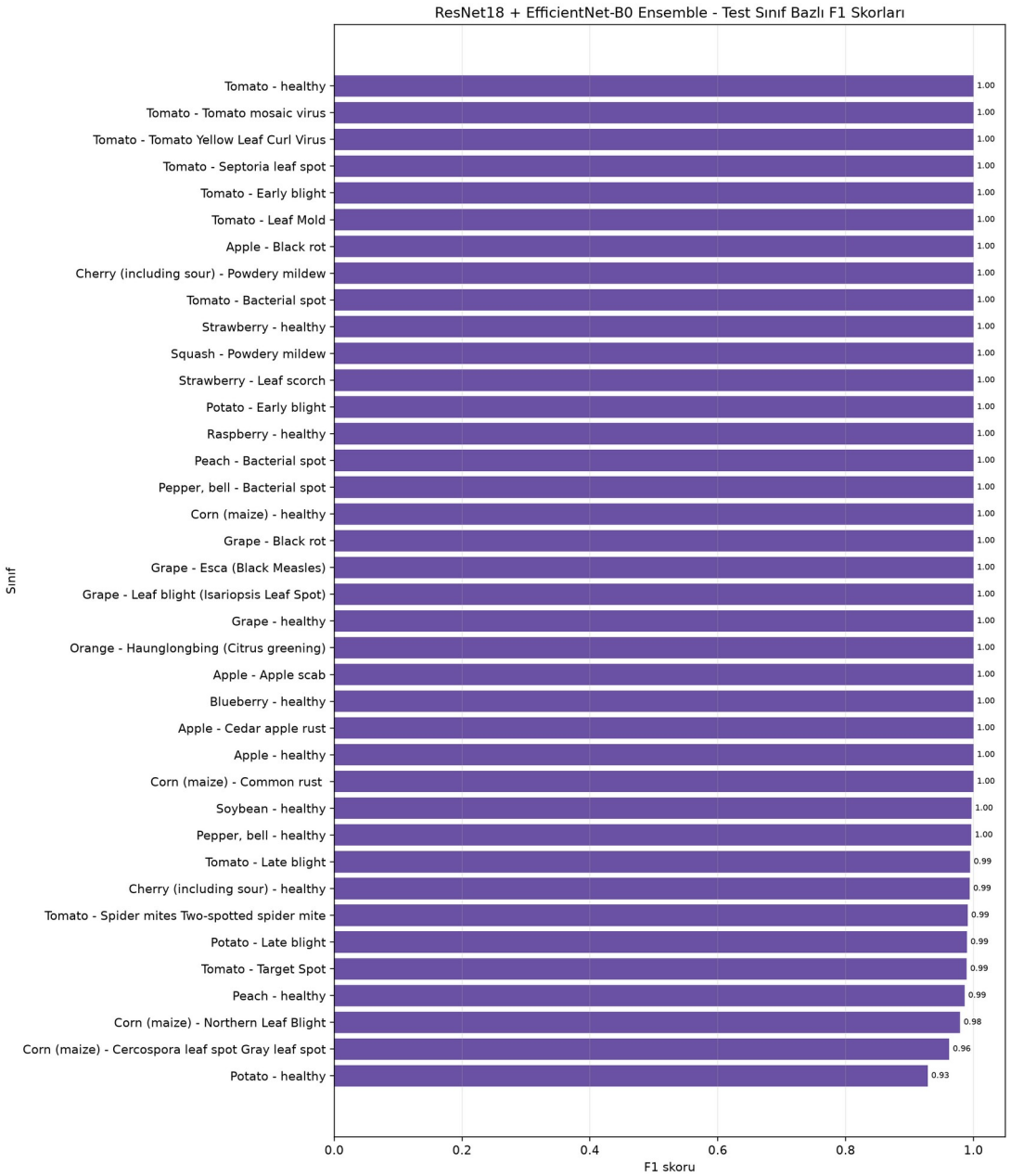

Figure 9. Ensemble per-class F1 scores. Potato healthy, Cercospora leaf spot, and Northern Leaf Blight remain the most difficult classes.

True class	Predicted class	Count	True-class error rate
Corn Northern Leaf Blight	Corn Cercospora/Gray leaf spot	4	4.08%
Tomato Spider mites	Tomato Target Spot	3	1.79%
Potato Late blight	Tomato Late blight	2	2.00%
Potato healthy	Soybean healthy	1	6.67%
Potato healthy	Pepper bell healthy	1	6.67%
Cherry healthy	Soybean healthy	1	1.18%

Table 11. Most frequent ensemble confusion pairs.

The lowest ensemble F1 score is 0.9286 for Potato healthy, whose test support is only 15 images. Corn Cercospora/Gray leaf spot reaches 0.9623 F1, and Northern Leaf Blight reaches 0.9792. Four Northern Leaf Blight images were labeled as Cercospora/Gray leaf spot, reflecting visually similar elongated maize lesions. The rarity of Potato healthy makes each error disproportionately important to its recall and F1. These results illustrate why accuracy alone would give an incomplete assessment.

Grad-CAM interpretation

Correct examples generally emphasize biologically plausible leaf regions: a small visible lesion in Apple black rot, broad venation and texture in healthy leaves, discolored tissue in Tomato leaf mold, and central texture changes in the Spider-mite example. This supports the interpretation that the model frequently relies on leaf evidence rather than only on the gray background.

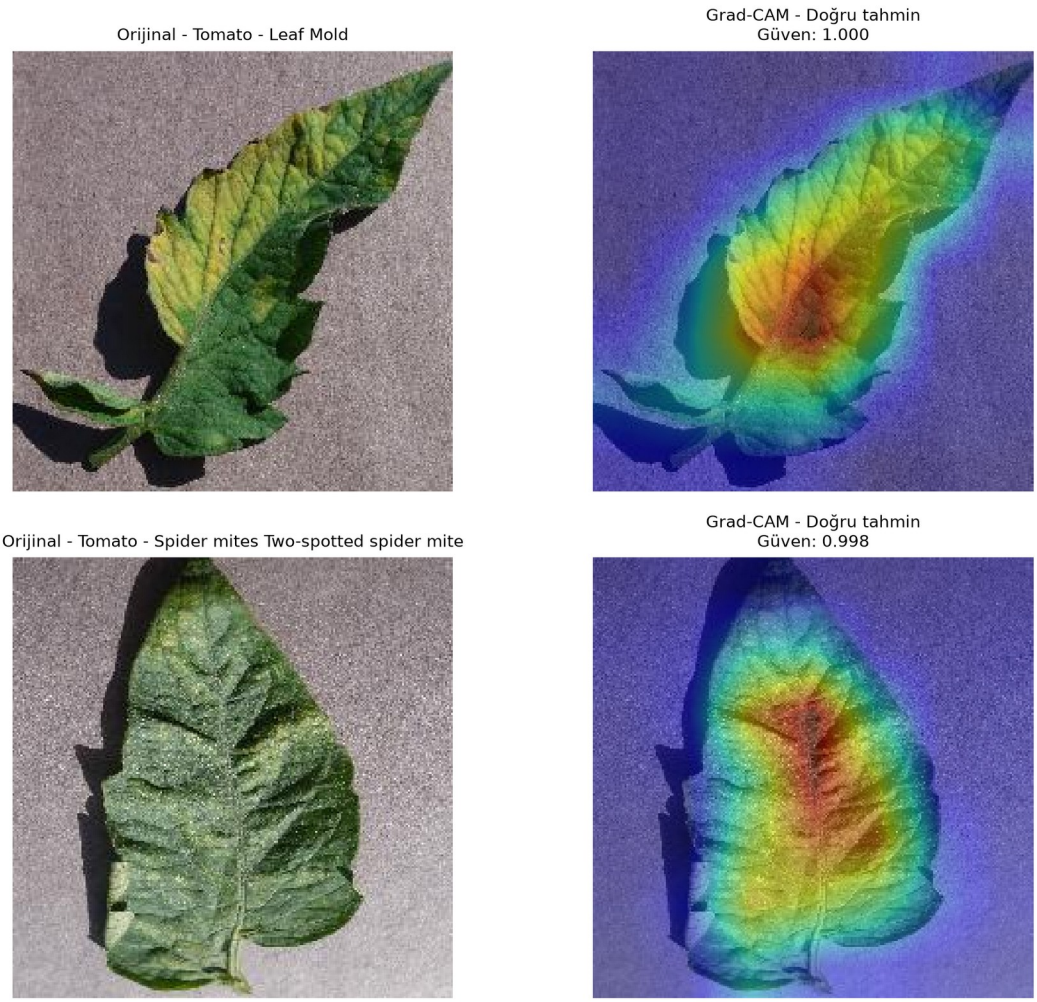

Figure 10. Two representative correct EfficientNet-B0 predictions from the six generated Grad-CAM cases. Warm colors indicate regions with stronger class-specific influence.

Error maps are also informative. The two maize disease classes attend to overlapping lesion structures; Tomato Spider mites, Target Spot, Leaf Mold, Bacterial spot, and Septoria can share small, distributed texture changes. The Potato healthy example was predicted as Pepper bell healthy with very high confidence, suggesting that leaf shape and healthy surface texture can dominate when class-specific disease evidence is absent. Some maps extend into shadows or background edges, revealing a possible dataset bias. Grad-CAM remains a coarse post-hoc explanation and does not establish causal feature use.

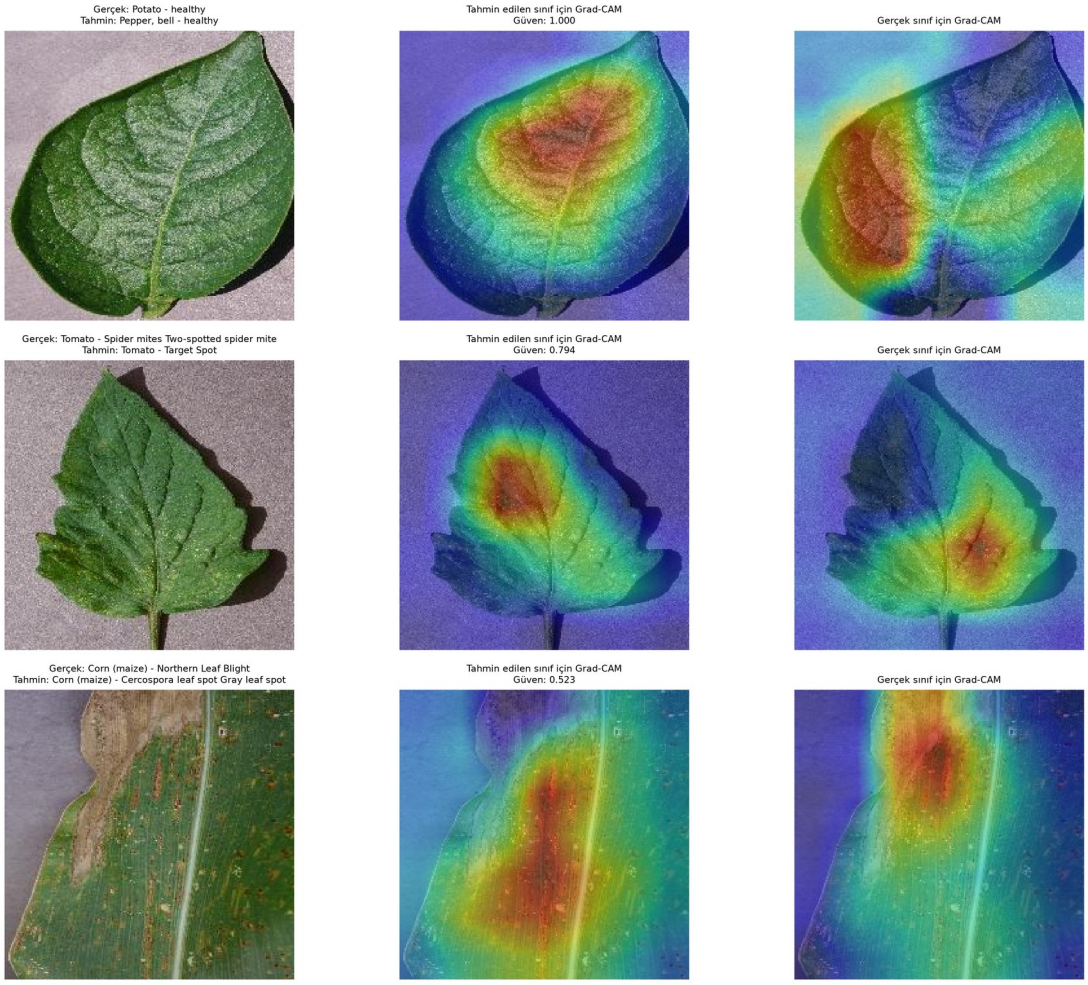

Figure 11. Three representative errors from the six generated Grad-CAM cases. Each row shows the original image, the predicted-class map, and the true-class map.

Limitations and Threats to Validity

Controlled imaging domain. PlantVillage images typically use simple backgrounds and isolated leaves. The evaluation therefore measures same-domain recognition rather than field robustness. Prior work reported a large drop on external images [3].

Image-level split assumptions. The stratified split prevents index overlap, but the available metadata do not guarantee plant-instance or acquisition-session separation. Near-duplicate views could make the test protocol easier.

Rare classes. Potato healthy has only 15 test examples; its class metrics have high sampling uncertainty. Macro-F1 limits domination by large classes but cannot eliminate uncertainty from small supports.

Limited ablation budget. Learning-rate pilots and a controlled three-rate dropout ablation were completed for the baseline, but augmentation-component ablations, class-weighting experiments, and frozen-versus-full fine-tuning comparisons were not completed. The dropout study used only three epochs and one fixed seed; therefore the small 0.20-versus-0.40 difference should be interpreted as sensitivity evidence rather than a definitive optimum.

Single random seed. The deterministic seed improves reproducibility, but confidence intervals over multiple independent training runs were not estimated.

Ensemble deployment cost. The best aggregate score requires two models and therefore increases inference latency, memory, and operational complexity.

Conclusion and Future Work

This project implemented a complete plant-disease classification workflow rather than relying on a ready-made notebook. The custom CNN established a transparent 399k-parameter baseline, while full fine-tuning of ImageNet-pretrained networks produced a large performance gain. EfficientNet-B0 was the strongest single model, reaching 99.52% test accuracy and 0.9923 Macro-F1 with only 4.06 million parameters. ResNet18 remained highly competitive at 99.26% accuracy. Validation-tuned equal-weight soft voting combined their complementary outputs and reached 99.76% accuracy, 0.9950 Macro-F1, and only 13 errors across 5,431 test images. Grad-CAM provided qualitative evidence that the single-model predictions often rely on lesions, texture, and venation, while also revealing occasional background and shape dependence. The dropout sensitivity study showed that $p = 0.20$ and $p = 0.40$ were similar in the three-epoch baseline pilot, while $p = 0.60$ was substantially weaker, consistent with excessive regularization for the short training budget. Early stopping remained implemented as a validation-Macro-F1 safeguard, but the final fixed-length runs reached their epoch caps before its patience criterion fired; therefore no claim is made that it shortened those final runs.

Future work should evaluate the models on field photographs, enforce plant-instance-aware splits where metadata permit, report multiple-seed confidence intervals, and test class-balanced or focal losses for rare classes. A compact deployment study could compare the ensemble against EfficientNet-B0 using inference latency, energy consumption, and model compression. Domain adaptation, stronger background randomization, and a mobile interface would be natural extensions toward real agricultural use.

References

[1] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015. Available online

[2] M. Singh, “PlantVillage Dataset,” Kaggle. Accessed: July 30, 2026. Available online

[3] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” Frontiers in Plant Science, vol. 7, article 1419, 2016. Available online

[4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE CVPR, pp. 770-778, 2016. Available online

[5] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. ICML, PMLR 97, pp. 6105-6114, 2019. Available online

[6] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. ICML, PMLR 37, pp. 448-456, 2015. Available online

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” JMLR, vol. 15, pp. 1929-1958, 2014. Available online

[8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. ICLR, 2019. Available online

[9] R. R. Selvaraju et al., “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE ICCV, pp. 618-626, 2017. Available online

[10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32, pp. 8024-8035, 2019. Available online

[11] J. Deng et al., “ImageNet: A large-scale hierarchical image database,” in Proc. IEEE CVPR, pp. 248-255, 2009. Available online